

```

1 import serial
2 import smtplib
3 from email.mime.text import MIMEText
4 from email.mime.multipart import MIMEMultipart
5 from email.mime.base import MIMEBase
6 from email import encoders
7
8 # serial communication port should be changed according to your own Board's COM port
9 ser = serial.Serial('COM9', 9600)
10
11 # email configuration
12 email = " " #here we have to add sender's email address
13 password = " " #here we have to add sender's email address password
14 recipients = " " #here we have to add reciever's address
15
16 def send_email(subject, message, attachment=None):
17     msg = MIMEMultipart()
18     msg['From'] = email
19     msg['To'] = ", ".join(recipients)
20     msg['Subject'] = subject
21     msg.attach(MIMEText(message))
22
23     if attachment:
24         attachment_file = open(attachment, 'rb')
25         part = MIMEBase('application', 'octet-stream')
26         part.set_payload(attachment_file.read())
27         encoders.encode_base64(part)
28         part.add_header('Content-Disposition', "attachment; filename= %s" % attachment)
29         msg.attach(part)
30
31     server = smtplib.SMTP('smtp.gmail.com', 587)

```

Fig. 5: Python code setting the sender's email, password, and receiver's email



Fig. 6: OLED display showing value

Generating an app password

To facilitate email sending, specific settings must be configured, including generating a third-party app password. This ensures security and successful transmission of emails.

Turn on 2-step verifi-

cation for your Google account:

- Go to your Google account and select Security.
- Under 'Signing into Google,' select the 2-Step Verification and follow the on-screen steps.
- Scroll down and select App passwords.
- Enter a name to identify where you'll use the app password.
- Select Generate and follow the on-screen instructions.
- The app password will be a 16-character code that appears on your device.
- Select Done to complete the process.

Now you are ready to proceed with the rest of the steps in your project.

Construction and testing

The construction process begins by uploading the source code onto the Arduino Uno. Following this, the circuit is assembled on a breadboard according to the wiring diagram provided. Connecting the device to a laptop, the Python code is executed after adjusting the COM port. Alcohol detection prompts the device to alert through the buzzer, and subsequently an email is dispatched to the pre-set address, complete with the detected location. The vehicle's speed can be observed on the OLED display.

Fig. 6 shows the speed of vehicle on OLED while Fig. 7 shows screen shot of the email sent from the device. **EFY**

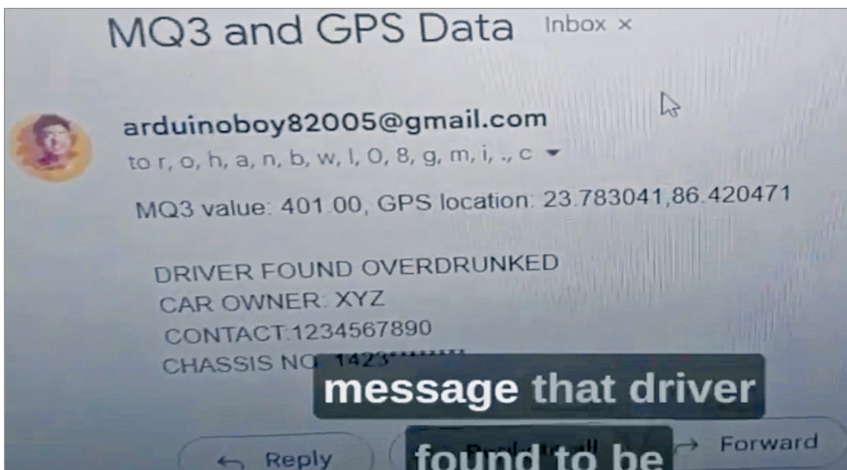


Fig. 7: An email sent from the device



Rohan Barnwal is an enthusiastic professional with a passion for IoT and electronics